Human Verification: `Containers.Thin`

Polytrees and Thinness of the Network Directed Container

Machine-generated from `lean/Thin.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Thin` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 Reachability and path structure **2**
- 2.1 Definition: `Digraph.Reachable` . . . 2
- 2.2 Definition: `Path.isTrivial` . . . 2
- 2.3 Definition: `Digraph.Acyclic` . . . 2
- 2.4 Definition: `Digraph.EdgeDisjoint` . . . 3
- 2.5 Structure: `Digraph.Polytree` . . . 3

3 Auxiliary lemmas and path decomposition **3**
- 3.1 Lemma: `acyclic_no_cons` . . . 3
- 3.2 Inductive type: `PathView` . . . 4
- 3.3 Definitions and theorem: `Path.view`, `PathView.toPath`, `Path.view_toPath` . . . 4

4 Main results **5**
- 4.1 Theorem: `path_unique_of_polytree` . . . 5
- 4.2 Theorem: `thin_of_polytree` . . . 5

5 Position bijection **5**
- 5.1 Structure: `Equiv'` . . . 5
- 5.2 Definition: `Reachable.path` . . . 6
- 5.3 Definition: `endpoint_equiv` . . . 6

6 Translation conventions **7**

1 Overview

This file verifies Proposition *prop:tree* of the supply-chains paper: if G is a **polytree** (underlying undirected graph is a forest), then the directed container network(G) is **thin** — between any two nodes there is at most one directed path.

The main results are:

- `thin_of_polytree`: in a polytree, any two directed paths between the same endpoints are equal (thinness = path uniqueness).
- `endpoint_equiv`: positions at v in network(G) biject with reachable nodes from v.

All definitions and proofs are in Lean 4 core; no Mathlib.

2 Reachability and path structure

2.1 Definition: `Digraph.Reachable`

Definition 2.1 (Reachability)**.** A vertex w is **reachable** from v in digraph G if there exists a directed path from v to w:

$$\text{Reachable}(G, v, w) \iff \text{Nonempty}(\text{Path}(G, v, w)).$$

Lean.

```
def Reachable (G : Digraph) (v w : G.V) : Prop :=
  Nonempty (Path G v w)
```

Haskell.

```
type Reachable g v w = Bool    -- True iff there is a path from v to w
reachable :: Digraph g ⇒ g → Vertex → Vertex → Bool
reachable g v w = not (null (allPaths g v w))
```

2.2 Definition: `Path.isTrivial`

Definition 2.2 (Trivial path)**.** A path p is **trivial** (empty) when it was constructed by `nil`; a `cons` path is always non-trivial:

$$\text{isTrivial}(\text{nil}\, v) = \top, \qquad \text{isTrivial}(\text{cons}\, e\, p) = \bot.$$

Lean.

```
def Path.isTrivial : {v w : G.V} → Path G v w → Prop
  | _, _, .nil _ ⇒ True
  | _, _, .cons _ _ ⇒ False
```

Haskell.

```
isTrivial :: Path → Bool
isTrivial []      = True    -- nil path
isTrivial (_:_)   = False   -- cons path
```

2.3 Definition: `Digraph.Acyclic`

Definition 2.3 (Acyclic graph)**.** A digraph G is **acyclic** if every path from a vertex to itself is trivial (empty):

$$\text{Acyclic}(G) \iff \forall v : V,\ \forall p : \text{Path}(G, v, v),\ p.\text{isTrivial}.$$

Lean.

```
def Acyclic (G : Digraph) : Prop :=
  ∀ (v : G.V) (p : Path G v v), p.isTrivial
```

Haskell.

```
isAcyclic :: Digraph g ⇒ g → Bool
isAcyclic g = all (\v → isTrivial (shortestCycle g v)) (vertices g)
-- Equivalently: the graph has no directed cycles.
```

2.4 Definition: `Digraph.EdgeDisjoint`

Definition 2.4 (Edge-disjoint reachability)**.** A digraph G has **edge-disjoint reachability** if for any two distinct edges $e_1 \neq e_2$ with the same source, no vertex is reachable from both targets:

$$\text{EdgeDisjoint}(G) \;:\Longleftrightarrow\; \forall e_1, e_2 : E,\ \text{src}(e_1) = \text{src}(e_2) \Rightarrow e_1 \neq e_2 \Rightarrow \forall w,\ \neg(\text{Reachable}(G, \text{tgt}(e_1), w) \wedge \text{Reachable}$$

Lean.

```
def EdgeDisjoint (G : Digraph) : Prop :=
  ∀ (e1 e2 : G.E), G.src e1 = G.src e2 → e1 ≠ e2 →
    ∀ w, -(Reachable G (G.tgt e1) w /\ Reachable G (G.tgt e2) w)
```

Haskell.

```
edgeDisjoint :: Digraph g ⇒ g → Bool
edgeDisjoint g = and
  [ null (reachableSet g (tgt e1) `intersect` reachableSet g (tgt e2))
  | e1 <- edges g, e2 <- edges g, e1 /= e2, src e1 == src e2 ]
```

2.5 Structure: `Digraph.Polytree`

Definition 2.5 (Polytree)**.** A **polytree** is a digraph that is both acyclic and edge-disjoint:

$$\text{Polytree}(G) \;:\Longleftrightarrow\; \text{Acyclic}(G) \wedge \text{EdgeDisjoint}(G).$$

Lean.

```
structure Polytree (G : Digraph) : Prop where
  acyclic : Acyclic G
  edgeDisjoint : EdgeDisjoint G
```

Haskell.

```
data Polytree g = Polytree
  { acyclic      :: Bool   -- isAcyclic g
  , edgeDisjoint :: Bool   -- edgeDisjoint g
  }
```

3 Auxiliary lemmas and path decomposition

3.1 Lemma: `acyclic_no_cons`

Proposition 3.1 (Acyclicity excludes cons self-loops)**.** In an acyclic graph, if $e : E$ is an edge, there is no directed path from $\text{tgt}(e)$ back to $\text{src}(e)$:

$$\text{Acyclic}(G) \Rightarrow \neg\,\text{Path}(G,\, \text{tgt}(e),\, \text{src}(e)).$$

Lean.

```
theorem acyclic_no_cons (hac : Acyclic G) (e : G.E) (p : Path G (G.tgt
    e) (G.src e)) :
    False
```

Haskell.

```
-- In an acyclic graph, no back-edge path exists:
-- acyclic_no_cons :: Acyclic g → Edge → Path (tgt e) (src e) →
   Void
```

3.2 Inductive type: `PathView`

Definition 3.2 (Path view)**.** A **path view** from v to w is either:

- a **trivial** view with a proof $h : v = w$, or
- a **step**: an edge e with $\mathrm{src}(e) = v$, followed by a path $\mathrm{tail} : \mathrm{Path}(G, \mathrm{tgt}(e), w)$.

This packages the index equalities explicitly to enable clean pattern matching without triggering dependent-elimination issues.

Lean.

```
inductive PathView (G : Digraph) (v w : G.V) : Type
  | trivial (h : v = w) : PathView G v w
  | step (e : G.E) (he : G.src e = v) (tail : Path G (G.tgt e) w)
      : PathView G v w
```

Haskell.

```
data PathView v w
  = Trivial     -- v = w
  | Step Edge (Path w)    -- an edge followed by a path to w
```

3.3 Definitions and theorem: `Path.view`, `PathView.toPath`, `Path.view_toPath`

Definition 3.3 (View of a path)**.** Every path can be viewed as trivial or a step:

$$\mathrm{view}(\mathrm{nil}\, v) = \mathrm{trivial}(\mathrm{rfl}),$$
$$\mathrm{view}(\mathrm{cons}\, e\, p) = \mathrm{step}(e, \mathrm{rfl}, p).$$

Definition 3.4 (Path from a view)**.** Recover the original path from its view:

$$\mathrm{toPath}(\mathrm{trivial}\, h) = h \triangleright \mathrm{nil},$$
$$\mathrm{toPath}(\mathrm{step}\, e\, h_e\, p) = h_e \triangleright \mathrm{cons}(e, p).$$

Theorem 3.5 (View round-trips)**.** For any path p, $\mathrm{toPath}(\mathrm{view}(p)) = p$.

Lean.

```
def Path.view : {v w : G.V} → Path G v w → PathView G v w
  | _, _, .nil v ⇒ .trivial rfl
  | _, _, .cons e p ⇒ .step e rfl p

def PathView.toPath : {v w : G.V} → PathView G v w → Path G v w
  | _, _, .trivial h ⇒ h ▸ .nil _
  | _, _, .step e he p ⇒ he ▸ .cons e p

theorem Path.view_toPath : {v w : G.V} → (p : Path G v w) →
    p.view.toPath = p
```

Haskell.

```
viewPath :: Path v w → PathView v w
viewPath []     = Trivial
viewPath (e:es) = Step e es

toPath :: PathView v w → Path v w
toPath Trivial       = []
toPath (Step e es) = e : es

-- prop_view_roundtrip p = toPath (viewPath p) == p
```

4 Main results

4.1 Theorem: `path_unique_of_polytree`

Theorem 4.1 (Path uniqueness in polytrees)**.** In a polytree G, any two directed paths between the same endpoints are equal:

$$\text{Polytree}(G) \;\Rightarrow\; \forall v, w : V,\ \forall p, q : \text{Path}(G, v, w),\ p = q.$$

Proof strategy: structural induction on p, decomposing q via `PathView` to avoid dependent-elimination issues.

- `nil`/`trivial`: both paths are `nil`.
- `nil`/`step`: the step creates a directed cycle, contradicting acyclicity.
- `cons`/`trivial`: symmetric cycle contradiction.
- `cons` e_1 / `step` e_2: if $e_1 = e_2$, apply the induction hypothesis to the tails; if $e_1 \neq e_2$, edge-disjointness is violated.

Lean.

```
theorem path_unique_of_polytree (ht : Polytree G) :
    {v w : G.V} → (p q : Path G v w) → p = q
```

Haskell.

```
-- prop_unique :: Polytree g → Path v w → Path v w → Bool
-- prop_unique ht p q = p == q
--
-- In a polytree, all paths between the same endpoints are identical.
```

4.2 Theorem: `thin_of_polytree`

Theorem 4.2 (Thinness of the network container)**.** If G is a polytree, then for any two vertices v and w, the type $\text{Path}(G, v, w)$ is a subsingleton (has at most one element):

$$\text{Polytree}(G) \;\Rightarrow\; \forall v, w : V,\ \text{Subsingleton}(\text{Path}(G, v, w)).$$

This is the thinness condition on the directed container $\text{network}(G)$: between any two nodes there is at most one path.

Lean.

```
theorem thin_of_polytree (ht : Polytree G) (v w : G.V) :
    Subsingleton (Path G v w)
```

Haskell.

```
-- prop_thin :: Polytree g → Vertex → Vertex → Bool
-- prop_thin ht v w = length (allPaths g v w) <= 1
```

5 Position bijection

5.1 Structure: `Equiv'`

Definition 5.1 (Equivalence (Mathlib-free))**.** An **equivalence** between types α and β is a pair of inverse maps:

$$\text{Equiv}'(\alpha, \beta) \;=\; \{(f, g, l, r) \mid f : \alpha \to \beta,\ g : \beta \to \alpha,\ l : g \circ f = \text{id},\ r : f \circ g = \text{id}\}.$$

Lean.

```
structure Equiv' (alpha beta : Type) where
  toFun    : alpha → beta
  invFun   : beta → alpha
  left_inv  : ∀ a, invFun (toFun a) = a
  right_inv : ∀ b, toFun (invFun b) = b
```

Haskell.

```
data Equiv a b = Equiv
  { toFun    :: a → b
  , invFun   :: b → a
  -- left_inv:  invFun . toFun = id
  -- right_inv: toFun . invFun = id
  }
```

5.2 Definition: `Reachable.path`

Definition 5.2 (Choosing a path witness)**.** Given a reachability proof $h_r : \mathrm{Reachable}(G, v, w)$, choose a path by classical choice:

$$h_r.\mathrm{path} \;=\; \mathrm{Classical.choice}(h_r).$$

This is `noncomputable` (no constructive witness is extracted).

Lean.

```
noncomputable def Reachable.path {v w : G.V}
    (hr : Reachable G v w) : Path G v w :=
  Classical.choice hr
```

Haskell.

```
-- Non-constructive: pick any path given a reachability certificate.
-- In Haskell we would just compute one (e.g. BFS).
reachablePath :: Reachable → Path
reachablePath = head . allPaths  -- pick arbitrarily
```

Fidelity note. In Lean, `noncomputable` signals that the definition uses `Classical.choice` and cannot be extracted to code. In Haskell (or an actual implementation) a constructive path would be computed by BFS or DFS.

5.3 Definition: `endpoint_equiv`

Definition 5.3 (Positions biject with reachable nodes)**.** In a polytree G, positions at v in network(G) — which are pairs (w, p) with $p : \mathrm{Path}(G, v, w)$ — biject with reachable nodes from v:

$$\mathrm{endpoint_equiv}(\mathrm{ht}, v) \;:\; \left(\Sigma_{w:V}\, \mathrm{Path}(G, v, w)\right) \;\simeq\; \{w : V \mid \mathrm{Reachable}(G, v, w)\}.$$

- Forward: $(w, p) \mapsto (w, \langle p \rangle)$ (pack into a reachability proof).
- Backward: $(w, h_r) \mapsto (w, h_r.\mathrm{path})$ (choose the unique path via `path_unique_of_polytree`).

This is `noncomputable` (it uses `Classical.choice` via `Reachable.path`).

Lean.

```
noncomputable def endpoint_equiv (ht : Polytree G) (v : G.V) :
    Equiv' ((w : G.V) * Path G v w) {w : G.V // Reachable G v w} where
  toFun p := ⟨p.1, ⟨p.2⟩⟩
  invFun w := ⟨w.1, w.2.path⟩
  left_inv p := ...
  right_inv w := ...
```

Haskell.

```haskell
endpoint_equiv :: Polytree g → Vertex
               → Equiv (Vertex, Path) (Vertex, Bool {- reachable -})
endpoint_equiv ht v = Equiv
  { toFun  = \(w, p) → (w, True)
  , invFun = \(w, _) → (w, uniquePath v w)  -- unique by polytree
      assumption
  }
```

Fidelity note. The Lean statement uses a dependent Σ-type for positions and a subtype for reachable nodes. Haskell approximates these with pairs and a boolean flag. The key mathematical content — the bijection between paths and reachable nodes in a polytree — is preserved.

6 Translation conventions

1. **Classical choice.** `endpoint_equiv` uses `Classical.choice`; it is `noncomputable`. In a constructive or implementation setting, one would compute a path by graph search.
2. **Dependent types for positions.** Positions in network(G) are $\Sigma_w \, \mathrm{Path}(G, v, w)$; the endpoint w is *determined* by the path. Haskell approximates this with a plain pair.
3. **PathView.** The `PathView` type exists solely to avoid dependent-elimination failures when matching on the second path in the uniqueness proof. It has no mathematical content beyond providing a clean view of the path type.
4. **Proof bodies omitted.** Only statements are translated.